sas innovate

Quick Start to the SAS® Macro Language

Allison Saito
Sr Technical Training Consultant

#SASinnovate

Contents

Abstract **2**

Key Concepts from Slides **3**

Follow Along Instructions **5**

If following along in the hands-on workshop lab at SAS Innovate 2026 *5*

If following along in any other context *5*

Demo / Building a Macro **6**

Step 0: Make a plan *6*

Step 1: Write SAS code that has no macro components, but does the job for one set of inputs *6*

Step 2: Generalize with macro variables *6*

Step 2b: Document and clean up *7*

Step 3: Put code inside a macro definition *7*

Step 3b: Document and clean up *8*

Step 4: Add conditional processing to make the macro more robust *8*

Step 4b: Document and clean up *10*

Final Program **11**

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

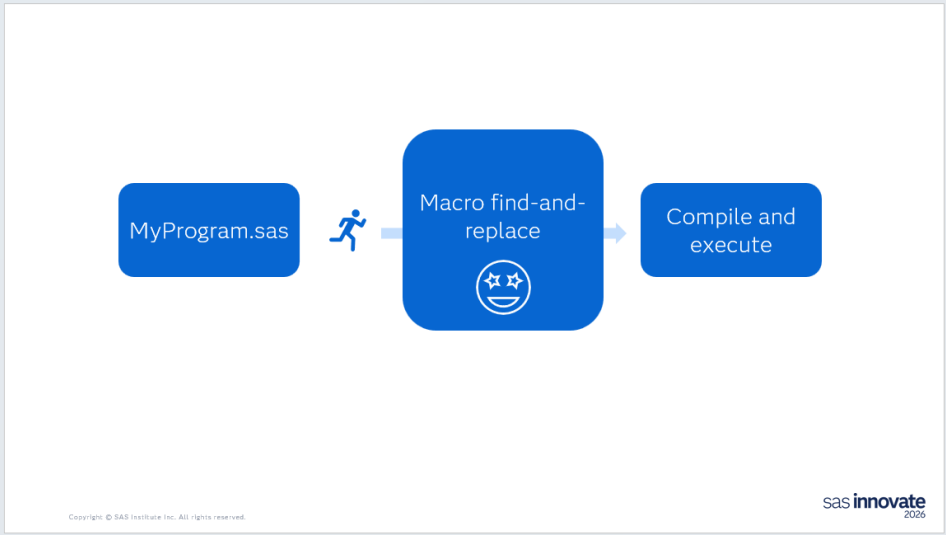
Abstract

Join this session to learn about SAS® Macro Language variables and programs and how they are processed.

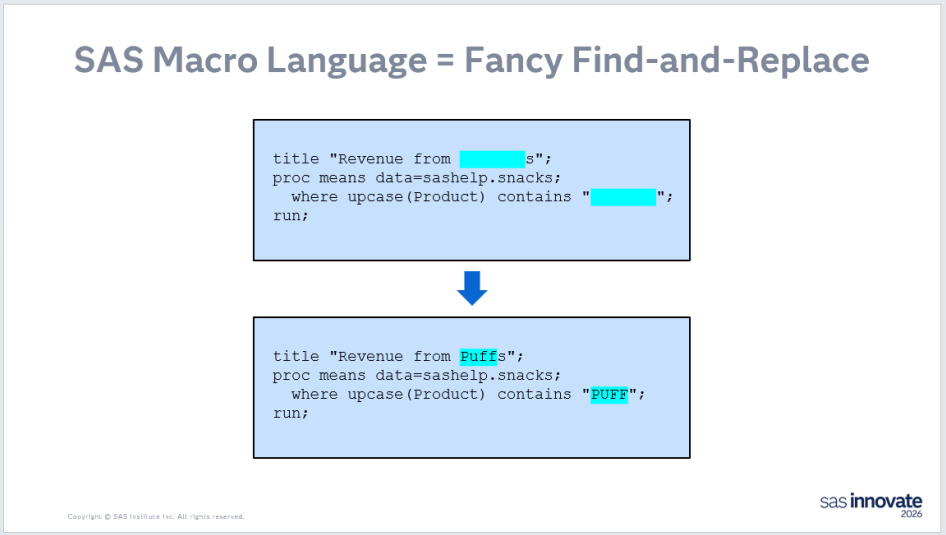
Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Key Concepts from Slides

- When using macro, there is a 'helper' between us and compile-and-execute
- The helper does fancy find-and-replace. Macro is the helper.



- Most of the macro facility's (AKA find-and-replace helper's) work is done before program execution



Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

- It is more than just 'find text A, and replace with text B'

Why 'fancy' find-and-replace?

- Replace text with ….
  - Given text
  - Text a function has been applied to
  - A value that was calculated by SAS
- Also …
  - IF/THEN/ELSE to control what text is input
  - Loops

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Follow Along Instructions

If following along in the hands-on workshop lab at SAS Innovate 2026

1. Open SAS Studio
    - Open Chrome
    - Use the bookmark for **SASStudio**
2. Sign in using the auto-filled username (*student*) and password
3. When asked "Do you want to opt in to all of your assumable groups?" click **No**. Nothing in this workshop requires admin rights
4. On the left-hand side, there is a column of icons. If there is no text next to the icons, click the two arrows at the bottom of the column. Descriptive text should now appear next to each icon
5. Select **SAS Server**
6. Expand **Files** > **Home** > **workshop** > **QuickStartMacro**
7. Open **Quick Start for Macro – Starter program.sas**
8. The presenter will go through the **Demo / Building a Macro** section below. In each section, add the code highlighted in yellow to your program

If following along in any other context

1. Download **Quick Start for Macro – Starter program.sas** from https://github.com/SASInnovate2026/Quick-Start-to-the-SAS-Macro-Language
2. Open it in your SAS programming interface of choice
3. Go through the **Demo / Building a Macro** section below. In each ection, add the code highlighted in yellow to your program

Commented [KS1]: Got an error that the page was not found

Commented [AS2R1]: Currently expected. I'll keep this comment active to remind me to check it closer to the date

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Demo / Building a Macro

Step 0: Make a plan

- Goal: Based on input variety, %snackGraph makes a temporary table for that variety of snack then graphs monthly revenue per product
- Input table: sashelp.snacks
- Output table: work.snacks

Step 1: Write SAS code that has no macro components, but does the job for one set of inputs

1. Open **Quick Start for Macro – Starter program.sas**
2. Confirm it accomplishes the goal for when *pretzel* is the variety. The output table should have 102 rows.

Step 2: Generalize with macro variables

1. Add a %LET statement to give **&variety** the value *PRETZEL*
2. Replace references to *pretzel* with the macro variable
3. Add macro functions to capitalize the final text appropriately
    - %SysFunc gets the macro facility to execute non-macro functions
4. Now, the macro facility effectively copies the macro variable's value (*PRETZEL*), and pastes it anywhere the macro variable is referenced (**&variety**). It also executes the macro functions to adjust the text
5. Run the code, and confirm the output is the same as in Step 1

Expected code

```
%let variety = PRETZEL;
proc sql;
create table work.snacks as
select put(date, yymmd.) as Month
   , product
   , sum(qtySold*price) as Revenue
        from sashelp.snacks
   where upcase(product) contains "%sysfunc(upcase(&variety))"
   group by Month
        , product
;
quit;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
title "Revenue from %sysFunc(propcase(&variety))s";
proc sgplot data=work.snacks;
     series x=Month Y=Revenue / group=Product;
run;
```

Step 2b: Document and clean up

1. Add any comments you think are needed to explain the code. Inside of macro definitions, the `/* in-line comment style */` works best
2. Remove any code that is no longer needed

Step 3: Put code inside a macro definition

1. Add a %macro statement at the beginning of the code you'd like the macro to control. Have it create a positional parameter named variety
2. Remove the %LET statement. The creation of the macro variable is now handled by the %macro statement's variable
3. Add a %mEnd statement after the end of the code you'd like the macro to control.
4. After the macro definition (code from the %macro to the %mEnd statement) is submitted to SAS, we can call it for different values of variety. Call %snackGraph(pretZEl) and %snackGraph(puFF). The irregular capitalization is intentional, to test the macro
5. To recompile the macro with the updated code, run from the %macro to %mEnd statement
6. Run the *pretZel* macro call. Confirm it gives the same output as steps 1 and 2
7. Run the *puFF* macro call. Confirm the output table and graph are made as expected

Expected code

```
%macro snackGraph(variety);

proc sql;
create table work.snacks as
select put(date, yymmd.) as Month
   , product
   , sum(qtySold*price) as Revenue
        from sashelp.snacks
   where upcase(product) contains "%sysfunc(upcase(&variety))"
   group by Month
        , product
;
quit;

title "Revenue from %sysFunc(propcase(&variety))s";
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
proc sgplot data=work.snacks;
     series x=Month Y=Revenue / group=Product;
run;

%mEnd snackGraph;

%snackGraph(pretZEl)
%snackGraph(puFF)
```

Step 3b: Document and clean up

1. Add any comments you think are needed to explain the code. It is strongly recommended to at least have a comment before the %macro statement that explains the purpose of the macro
2. Remove any code that is no longer needed
3. In some interfaces (such as SAS Enterprise Guide), you may have noticed a lot of text color coding went away. To trick the interface into giving back the color coding, add the following statement immediately after the %macro statement. Include a comment explaining what it does

```
      %macro a; %mEnd a;
```

Expected code

```
/*
 * %snackGraph
 *    Purpose: Summarize data to revenue per month, and subset to
 *             only products of a given variety
 *    Input: variety parameter (single word, used to subset
 *           products)
 *    Output: work.snack and a line graph of revenue, both
 *            summarized and subset
*/
%macro snackGraph(variety);
  %macro a; %mEnd a; /* restores color coding in some interfaces */

  proc sql;

...
```

Step 4: Add conditional processing to make the macro more robust

1. For this macro, if we put in a variety that does not exist in the data, no output is created and the log does not explicitly explain the issue. For example, if variety is *onion*, then the PROC SGPLOT creates a note in the log that 0 observations were read from the data set. Let's make the macro a little more user-friendly. If there are no snacks of the requested variety, we'll have the macro not attempt the SGPLOT and put a descriptive warning in the log.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

2. To do this, we'll need to check how many rows are in **work.snacks**, and then we'll determine the next action accordingly.
3. Fortunately, PROC SQL automatically created a macro variable named **&sqlObs** that contains the number of rows output by the most recent query. So, immediately after the PROC SQL step, **&sqlObs** contains the number of rows in work.snacks. Add the following %put statement to confirm the variable was made as described

```
%put &=sqlObs;
```

4. Add macro conditional processing to test if **&sqlObs > 0**. If it is, then run the PROC SGPLOT.
5. If **&sqlObs** is not greater than zero, use the following %put statement to put a warning in the log

```
%put WARNING: No rows have the value &variety in their
Product value.;
```

6. Add a call to %snackGraph(onion)
7. Recompile the macro (run from the %macro through the %mEnd statement), and run the macro calls. Confirm *pretZEl* and *puFF* still work as intended and that *onion* gives the appropriate warning in the log

Expected code

```
%macro snackGraph(variety);
%macro a; %mEnd a; /* restores color coding in Enterprise Guide */
proc sql;
create table work.snacks as
select put(date, yymmd.) as Month
   , product
   , sum(qtySold*price) as Revenue
        from sashelp.snacks
   where upcase(product) contains "%sysfunc(upcase(&variety))"
   group by Month
        , product
;
quit;

%put &=sqlObs;

%if &sqlObs > 0 %then %do;
title "Revenue from %sysFunc(propcase(&variety))s";
proc sgplot data=work.snacks;
     series x=Month Y=Revenue / group=Product;
run;
%end;
%else %do;
     %put WARNING: No rows have the value &variety in their Product
value.;
%end;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
%mEnd snackGraph;

%snackGraph(pretZEl)
%snackGraph(puFF)
%snackGraph(onion)
```

Step 4b: Document and clean up

1. Add any comments you think are needed to explain the code
2. Remove any code that is no longer needed

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Final Program

Note: Comments will vary

```
/*
 * %snackGraph
 *    Purpose: Summarize data to revenue per month, and subset to only
 *             products of a given variety
 *    Input: variety parameter (single word, used to subset products)
 *    Output: work.snack and a line graph of revenue, both summarized
 *             and subset
 */
%macro snackGraph(variety);
  %macro a; %mEnd a; /* restores color coding in some interfaces */

  proc sql;
  create table work.snacks as
  select put(date, yymmd.) as Month
           /* PUT function formats a value, then returns
             the formatted value as a string */
        , product
        , sum(qtySold*price) as Revenue
    from sashelp.snacks
    where upcase(product) contains "%sysFunc(upcase(&variety))"
           /* %sysFunc allows macro to use a standard function */
    group by Month
        , product
  ;
  quit;

  *% If zero rows in WORK.SNACKS, put warning in log.  Otherwise,
       graph it. ;
  %if &sqlObs = 0 %then %do;
    %put WARNING: No rows have the value &variety in their Product value.;
      /* puts warning into log */
  %end;
  %else %do;
    title "Revenue from %sysFunc(propcase(&variety))s";
             /* %sysFunc allows macro to use a standard function */
    proc sgplot data=work.snacks;
        series x=Month Y=Revenue / group=Product;
    run;
  %end;

%mEnd snackGraph;


%snackGraph(pretZEl)/* expect graph */
%snackGraph(puFF) /* expect graph */
%snackGraph(onion) /* expect warning in log */
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.